Morpheus on Lux

Zach Kelling

Lux Industries

2026

This document is the master record of a read-only evaluation of the Morpheus inference marketplace, conducted 2026-08-26 through 2026-08-29 against all 62 repositories in the `MorpheusAIs` GitHub organization and against live on-chain state on Ethereum, Arbitrum, and Base. No repository here is a fork of ours; no change was pushed anywhere. It is written for the Morpheus core team and their investors. Numbers carry a provenance label: [verified] read line by line from source or chain, [derived] computed from verified inputs, [illustrative] assumed for a worked example and not measured, [pending] not yet true and conditional on a stated event.

Contents

1 Executive summary

Morpheus describes itself as decentralized, verified, and agentic. The deployed code is careful engineering at the layers where most projects are weak, and it does none of those three things at the layer where it counts. This is not an accusation. Every gap named below is disclosed in Morpheus's own code, comments, or configuration, which by any honest standard makes it immaturity rather than deception. The point of the evaluation is narrower and more useful: each of the words in the marketing already maps to a system we run in production on Lux, Hanzo, or Zoo, so the distance between what Morpheus says and what Morpheus enforces is a distance we can close with code that already exists.

The core finding is one sentence. A provider is paid `pricePerSecond` times elapsed seconds (`SessionRouter.sol:264` [verified]), with no reference in the payment calculation to the model served, the tokens produced, or whether any inference occurred, and no slashing anywhere in the contract tree (a `grep` for `slash|penalt|forfeit` over the diamond returns nothing [verified]). The receipt the provider signs already carries `inputTokens` and `outputTokens` (`morrpcmessage/abi.go:12-20` [verified]), and the contract decodes five of its seven fields and drops both counts (`SessionRouter.sol:236-240` [verified]). Settlement is therefore fakeable, and every economic mechanism built on top of it, including the buyback that is supposed to give the token value, inherits that defect.

The economics around it are legible and unflattering. MOR trades at $1.95, a market capitalization near $17M, with 24-hour volume of $8,905 [verified]. Tradeable DEX liquidity is $1,411,470, of which the protocol itself owns 88.8% [verified]. Real external revenue, the Lido and Aave yield skimmed from deposits, was about $476K over the trailing four quarters and is falling [verified]; MOR emission over the same period is valued near $5M to $6M [verified]. Emission outruns revenue by roughly ten to one. The schedule that produces it (14,400 MOR/day declining to zero in 2040, 42,000,000 terminal, no premine [verified]) is the best property the token has, and the buyback that answers the dilution (about $511K over the trailing year [verified]) is a sound loop that is simply outrun.

The thesis follows directly. Keep MOR thin. Morpheus does not need to become a chain to become real; it needs the settlement it already advertises and a source of value that scales with usage rather than with deposits. The heavy machinery, post-quantum consensus, verified inference, threshold custody, a learned router, order-book liquidity, self-repaying credit, governance with a timelock, already runs on our stack. MOR taps it. It does not rebuild it.

The path is three phases with gates between them. Phase 0 stays on Base with no migration and is reversible: four one-transaction fixes, a verified-settlement adjunct, and a demand source. Phase 1 adds governance, an order-book venue, a bank rail, self-repaying credit, and a native bridge. Phase 2, only if volume justifies it, moves settlement onto a Lux L1 with MOR bridged as gas, introduces a value token, and cuts emission. Each phase turns on demonstrated on-chain revenue, not on a promise.

The discipline that governs this document is stated once and kept throughout. We do not claim what code does not enforce. That rule applies to our own stack, and Section 8 carries our punch-list in the first person, unedited, because a pitch that Morpheus should adopt our stack "to become real" is dishonest until our own stack meets the standard we are selling. Where we are early, we say early.

2 Findings

Four categories. Each finding carries an identifier, the evidence in source or on chain, and a severity we have deflated rather than inflated. The detailed write-ups live in the security assessment and the decentralization study; this section is the verified summary and the credit where it is due.

2.1 Security and control

CTL-1, Critical. A 5-of-9 multisig with no timelock has unilateral, instant, unbounded control over mint, upgrade, and treasury. Every capital contract is UUPS with `_authorizeUpgrade` gated only by `onlyOwner` (`DistributionV6.sol:776`, `DepositPool.sol:761`, and the same pattern across the tree [verified]). The token has no supply cap; `mint` requires only `isMinter[msg.sender]`

and `updateMinter` is `onlyOwner` (`MOROFT.sol:50,61` [verified]). The owner is one Gnosis Safe, `0x1FE04BC15Cf2c5A2d41a0b3a96725596676eBa1E` [verified]. Five keyholders acting together can mint arbitrary MOR, upgrade the pool that custodies about $20M of staked stETH [verified], or retarget the emission stream, in a single block, with no delay in which a depositor could observe and withdraw. Every other high-severity item reduces to "the owner does X." This one is the root, and it is fixable in one transaction: route ownership through a timelock.

CTL-2, High. The owner directs 52% of emission with no on-chain contribution check (`manageUsersInPrivatePool`, `DistributionV6.sol:175`, `onlyOwner` [verified]); the contributor-to-weight mapping lives in a document, not on chain.

SUP-1, High. The 42,000,000 figure is a curve parameter in the reward math, not a token invariant. The token itself has no ceiling; the same owner can add a minter and mint without bound. The "capped supply" property is enforced by convention, not by the contract.

2.2 Settlement and verifiability

SET-1, Medium. Settlement pays for elapsed time and reads nothing about the work delivered. `modelId` rides on the bid and never enters the bill (`SessionRouter.sol:264` [verified]). A provider is paid the same for the advertised model, a cheaper substitute, or a degraded quantization.

SET-2, Medium. The two signed token counts are dropped at the contract boundary. The receipt signs seven fields; `_extractProviderReceipt` decodes five (`SessionRouter.sol:236-240` [verified]). The gap is exactly `inputTokens` and `outputTokens`. The protocol discards, at no saving, the one authenticated work record it already collects. The fix reads two words.

SET-3, Medium. The ranked party signs its own ranking input. `tps` and `ttft` are accumulated by the provider's own `proxy-router` and feed a five-term scorer, none of which measures answer correctness (`rating/scorer_default.go:21-25` [verified]).

SET-4, Medium. Receipt validation checks the signer, not the counterparty, and has no self-dealing check; nothing requires consumer and provider addresses to differ (`SessionRouter.sol:537` [verified]). Bounded by the 1% daily budget, so a leak, not a drain.

SET-6, Informational. Attestation proves an enclave ran, not which weights ran or that a token reached the user. This is a property boundary, stated so the threat model is explicit: attestation is not inference verification.

2.3 Economics

ECO-1, Medium. Emission outruns real revenue by roughly ten to one, and the gap widens as price falls. Real yield was about $476K trailing year and falling steeply, from $3.24M in Q1 2024 to $63K in Q3 2026 [verified]; emission is valued near $5M to $6M/year [verified]. A lower token reduces the deposit incentive, which reduces yield, which reduces the buyback that supports the token. That feedback runs the wrong way.

ECO-2, Medium. Reselling third-party API quota is a documented provider path (`docs/providers/resale/reselling-venice.mdx` [verified]), and nothing on chain distinguishes a resold key from GPU compute.

ECO-3, Low. No slashing exists; provider downside is a temporary dispute-hold. A misbehaving provider risks time, not capital.

ECO-4, Informational. The yield-funded buyback (about $511K trailing year, 75% of captured yield to buy-and-burn, buy-and-lock, and protocol-owned liquidity [verified]) is well designed and sound. Its only problem is scale.

2.4 Decentralization and scale

Decentralization is graded as a vector over six powers (upgrade, custody, halt or censor, governance, exit, liveness), where the real score is the lowest load-bearing power, not an average. Morpheus today scores 0 on upgrade, custody, halt, and governance: one Safe can swap the compute diamond in one block, self-add as minter, recut settlement, and there is no on-chain vote surface (`MOROFT.sol` is a LayerZero OFT with no vote checkpoints; the MRC process is 86 Markdown files that bind nothing, dormant since 2025-11-17 [verified]). The honest counterweight is in Section 8: our own stack does not yet clear this bar either, and we say where.

On scale, the hosted gateway funds every Web2 caller from one shared consumer wallet; after a nonce conflict on a session open or close the path serializes, and throughput clears a few sessions per second [verified]. That is an architectural ceiling, not a tuning knob.

The concentration and liquidity facts, read from chain on 2026-08-29 and presented as risks to depositors rather than as accusations of intent. Existing supply is 9,286,960 MOR (Base 63%, Arbitrum 36%, Ethereum under 1%) [verified]. Price was $1.95, market capitalization $16.96M, 24-hour volume $8,905, down 63.9% on the year and 98.6% from the all-time high [verified]. The Compute Reserve multisig holds roughly 2.87M MOR, 30.9% of all MOR [verified]; the top ten holders on Base are 93.6% of Base supply [verified]. Tradeable DEX liquidity is $1,411,470, of which the protocol owns 88.8%, with a protocol-owned-liquidity Safe holding 99.98% of the main pool's active liquidity [verified]. Daily volume is 0.63% of liquidity; a $100,000 sell equals about eleven days of total market volume [derived]. Price discovery and exit both depend on liquidity the protocol controls. This is a fragility any depositor should be able to see stated plainly, and it is one a governance and liquidity plan (ADOPT-03, ADOPT-06) should address.

2.5 What Morpheus built well

This is not throat-clearing; these are the parts we would keep if we were building on their code, and Section 7 says so in detail.

- **The attestation engineering is serious.** 5,439 lines of Go bind a TDX `report_data` to a TLS certificate digest and check an NVIDIA `eat_nonce`, with RTMR3 computed in CI against a signed manifest [verified]. Most marketplaces claim confidential compute and ship nothing; Morpheus shipped a working measurement path.
- **The treasury bifurcation did its job.** Consumer-funded and treasury-funded payments take different code paths, and the 1% daily budget cap bounded the July 2026 stipend leak to roughly 25,000 MOR/day rather than draining the reserve [verified].
- **The receipt already carries the work record.** Seven signed fields, including both token counts [verified]. The harder half of verifiable settlement, getting a signed structured record off the provider, is already built.
- **Accounting is receipt-derived and the audit trail is real.** No premine; the `Docs/Security Audit Reports/` directory holds genuine Code4rena, Zenith, and Renascence reports [verified]. This is a project that paid for adversarial review and kept the receipts.

The Ponzi accusation, tested directly, is refuted. Depositor principal is withdrawable 1:1 (`DepositPool.sol:265` [verified]) and never redistributed; rewards are freshly minted MOR, structurally a block reward. The defensible statement is ECO-1: an inflationary reward under-earned by real demand, which is a sustainability question, not fraud.

3 The proposal, as a menu

The response is thirteen one-page proposals, the ADOPT series, each standing for a longer paper and each independently readable. They are organized into four tiers by dependency depth, not by importance. A tier is a floor: everything in it stands on what is below it. Morpheus can take one tier, or one paper, or all four. Nothing here is a bundle that fails unless bought whole.

Docket	What it fixes	Tier	Depends on
ADOPT-09	CTL-1 timelock, SET-2/5 decode and bind, SET-4 self-deal	1	none
ADOPT-02	SET-1/2/3, ECO-3: pay for work, not time	1	09
ADOPT-07	ECO-1 demand, ECO-2 resale, SET-6 attested supply	1	02
ADOPT-03	CTL-1/CTL-2: timelock, vote surface, owner emission	2	09
ADOPT-08	ECO-1/ECO-4: buyback fed by volume, not deposits	2	02
ADOPT-05	fiat on-ramp: accounts and cards	2	none
ADOPT-06	thin liquidity: order book and price discovery	2	03
ADOPT-04	idle $20M stETH, exit pressure: LMOR credit	2	06
ADOPT-13	hand-replicated cross-domain schedule: native bridge	3	04
ADOPT-10	demand loop: reciprocal routing across the group	3	07, 08
ADOPT-01	no chain, validator set, or consensus of its own	4	03, 13
ADOPT-12	keep MOR thin: tap, do not rebuild	4	01
ADOPT-11	few sessions per second: settlement throughput	4	01

Tier 1, on Base, no migration, reversible. The four one-transaction fixes protect depositors and cost nothing structural (ADOPT-09). Verified settlement makes the meter read the token counts the receipt already signs (ADOPT-02). A demand source and attested supply make the meter worth reading (ADOPT-07). This tier closes CTL-1, SET-1 through SET-5, ECO-3, and half of SET-3 without moving a single asset off Base.

Tier 2, value and governance. A timelock and a vote surface move control off the bare multisig (ADOPT-03). A fee on verified volume funds the buyback from usage instead of deposits (ADOPT-08). An order-book venue deepens a book that is 88.8% protocol-owned (ADOPT-06). Self-repaying credit puts the idle stETH and MOR to work without a forced sale (ADOPT-04). A bank rail brings fiat-native consumers in (ADOPT-05, the smallest dependency in the set).

Tier 3, cross-domain. A native MPC bridge replaces the schedule that is replicated by hand across the capital and compute domains (ADOPT-13). A reciprocal routing loop turns the group's own inference demand into MOR volume (ADOPT-10).

Tier 4, a chain of its own, only if warranted. A Lux L1 or L2 gives Morpheus the consensus, validator set, and finality it does not have today, inherited rather than built (ADOPT-01). MOR stays thin on top of it (ADOPT-12). Settlement throughput stops being a few sessions per second (ADOPT-11). This tier is optional and gated on volume; Section 9 states when it is worth the migration and when it is not.

4 Where MOR goes long term

MOR today is one token doing three jobs. It is emitted on a schedule, escrowed to open sessions, and paid to providers. Any token-evolution story has to fix settlement before the emission problem is worth discussing, because every value mechanism sits downstream of the meter.

4.1 Two tokens, and what gives the second one value

Split the roles. MOR stays the inflationary budget-and-rewards unit: emitted, spent as gas and escrow, paid to providers. xMOR becomes the low-velocity store of value. The question that decides whether xMOR is real or decorative is simple: what claim does an xMOR holder have that a MOR holder does not?

Burn-and-mint is the weaker construction. Burn MOR to mint xMOR at some ratio. If the ratio is fixed and reversible, xMOR is a wrapper and accrues nothing; if one-way, it is MOR minus an exit, worth less rather than more, unless xMOR separately carries a right. The mint ratio is a governance parameter, and a governance-set conversion between two pools of value is exactly the surface the same multisig that directs 76% of emission can capture. Burn-and-mint values throughput, not savings.

Share-token, in the xSUSHI style, is sounder. Stake MOR into a pool; xMOR is the share. Protocol fees buy MOR and add it to the pool, so each xMOR redeems for a growing quantity of MOR. Value is an endogenous cash-flow claim, not a parameter: the xMOR/MOR ratio rises

monotonically with fees collected, and there is no conversion ratio for anyone to set. Its one trust point is that fees actually reach the pool, which every honest design shares. Its only leakage is timing, deposit before a distribution and withdraw after, and streaming the reward over a vesting window collapses that arbitrage.

Pick the share-token, and state the thing the client should hear. If fees buy MOR and burn it, every MOR holder benefits equally and no second token is needed at all. The only defensible reason to mint xMOR is velocity separation, keeping the budget churn from diluting the balances people hold for value. That is a real reason. It is not a magic one.

4.2 Burning does not create value

Say it plainly. Price is market capitalization divided by supply, so shrinking supply lifts price only if market capitalization holds. Burning adds nothing to market capitalization. A deflationary token with no demand is the same pie cut into fewer slices. Burn is real value-accrual in exactly one case: when it is financed by fees that real users paid for real usage. Then the fee is the value entering the system and the burn is merely how that value reaches holders, as scarcity instead of as a dividend. A burn financed by emission or token sales is circular: emit, sell, buy back, burn, net zero minus slippage.

This is where our own discipline bites. Settlement is fakeable today, so a fee on unverified volume is a fee on a number a provider can fabricate, and a self-dealing provider will wash inference to drive a burn that lifts a token he holds. The burn is only as honest as the meter behind it. Verified settlement (ADOPT-02, and `lux/mord` at about 3,400 verified sessions/sec/core [verified], a settlement-rate figure, not network TPS) is the precondition, not a companion feature.

4.3 Utility is the engine

Volume comes from MOR being the unit of account for agentic compute: agent budgets, MCP-server calls, inference and compute generally. Every call spends MOR, and a fee on each call funds the buy-and-burn. Define the variables. Let E be emission in MOR/day, A the number of fee-bearing calls/day, c the average revenue per call in USD, f the fee fraction routed to buy-and-burn, and P the MOR price in USD. MOR burned per day is $B = fAc/P$. Net supply change is $E - B$, and the system is net-deflationary when

$$B > E, \qquad \text{that is,} \qquad Ac > \frac{EP}{f}.$$

The right side is the emission's daily dollar value divided by the fee rate. Everything is legible from there.

A worked example, verified inputs and an illustrative conclusion. Emission's daily dollar value is $EP \approx 14{,}400 \times \$1.95 \approx \$28{,}080$/day [derived]. At $f = 1\%$, net-deflationary needs gross fee-bearing volume $Ac > \$2.81$M/day, about \$1.0B/year [illustrative]. At $f = 5\%$, above \$205M/year; at $f = 10\%$, above \$102M/year. Against today's roughly \$476K/year of revenue, which is Lido and Aave yield rather than usage, that is the true size of the ask. The assumptions (constant price, fee taken in USD-equivalent and fully burned, all volume verified) do not hold exactly; the equation shows the shape, not a forecast.

4.4 Cutting emission moves the same inequality

E is not a protocol constant needing a hard fork. It is set by owner-controlled pool configuration today (`manageUsersInPrivatePool`, `DistributionV6`, `onlyOwner` [verified]). Lower E and the crossover bar $Ac > EP/f$ falls proportionally. Halve emission to 7,200/day and the 1% requirement halves to about \$510M/year; cut it tenfold and it falls to about \$102M/year [illustrative]. Emission reduction and a usage-driven burn are the same lever pushed from two ends. The honest footnote is that the same owner power that can cut emission is the centralization CTL-1 and CTL-2 flag; the ability is real, and so is the risk, which is why ADOPT-03's timelock precedes any emission change.

4.5 The ICO funds the transition, not the demand

On `lux.exchange`, conditional on SEC approval of the proposed regulation [pending]. Its job is price discovery, treasury, and protocol-owned liquidity, not a substitute for demand. The float is thin

(depth about $1.4M, 88.8% protocol-owned, 24h volume about $8.9k [verified]), and a raise deepens the book and funds the build-out of the fee-bearing utility. It does not manufacture usage. The raise should clear only into demonstrated on-chain revenue. Selling the token before the meter is honest and the utility exists is selling the thing we just called theater.

4.6 What can go wrong

Two-token models add complexity and a conversion surface someone can capture; prefer buy-and-burn on MOR unless velocity separation earns the second token. A burn with no demand is theater, and a burn on unverified volume is worse. Emission cuts take income from the recipients of the 76% owner-directed emission, who will resist. The ICO needs real demand first or it prices thin float into a spike that round-trips. And every appreciation claim here is downstream of one unshipped thing: verified settlement. Ship the meter, then the fee, then the burn, then the token that holds its value. In that order.

5 Listing and liquidity

ADOPT-06 named the venue problem and the order-book fix: a book that is 88.8% protocol-owned, where a $100k sell is eleven days of volume, and where a two-sided book replaces a passive reserve [verified]. This section is the layer above the venue. It is about how MOR reaches a liquid market, what kind of market that is, and how far that market extends. Three moves, ordered by how much review each needs: list MOR, bridge and lend against it, and carry the broader cross-listed market above it.

5.1 MOR as a first-class Lux asset

MOR is `Offering: unregulated` in the asset taxonomy [verified: asset-class.ts], so none of this touches the securities surface. Three wirings, all crypto, all shippable now:

- **Featured listing.** The interface's token list pins the ecosystem set — MOR alongside AI, ZOO, and LUX — at the top of the surface a user sees first. This is a one-file change at a single chokepoint, and the four are defined once as a typed `FeaturedToken` list whose class field refuses a security by construction. [verified: pkgs/lx TokenSelector suggested-tokens hook; `FeaturedToken` in pkgs/exchange/src/featured.ts]
- **Native bridge.** MOR is a LayerZero OFT on Ethereum, Arbitrum, and Base [verified]. Lux's own bridge already carries Arbitrum and Base, so MOR enters the set it already spans. Adding MOR is asset registration — three token entries plus an on-chain allowlist — and an L-token wrapper; no new chain integration. [verified: lux/bridge token registry, chain manifests, and `allowedTokens`]
- **LMOR.** MOR as collateral in `lux/liquid` self-repaying lending, so a holder borrows against MOR without a forced sale — the ADOPT-04 move applied to MOR itself, and the concrete form of "put the treasury's idle assets to work." Two paths: a Morpho market is one transaction and gives overcollateralised MOR credit today; the self-repaying market is the ADOPT-04 design and needs a yield-bearing MOR to retire the debt. [verified: lux/liquid — Morpho `createMarket` vs the Alchemix-style Liquid market]

What this needs from Morpheus: the MOR OFT address on each chain, and confirmation of the AI, ZOO, and LUX addresses. Nothing here moves a Morpheus contract or needs the 5-of-9.

5.2 The interface is a renderer, not a broker

The venue above carries more than MOR, and the model that lets it do so cleanly is already the one our interface is built on. It converts a user's chosen parameters into a transaction the user signs in their own wallet. It never holds a key, never executes or settles, never routes an order, and is paid only by the user [verified: COMPLIANCE.md]. Under the SEC Division of Trading and Markets staff statement of 13 April 2026 (File No. 4-894), a provider of such an interface — a Covered User Interface Provider — is not a broker or dealer under Exchange Act section 15, provided it meets the statement's enumerated conditions. Our `COMPLIANCE.md` is written against that statement directly [verified].

Stated plainly, because a design built on it inherits its expiry: the statement is staff views with

no legal force, withdrawn 13 April 2031 absent Commission action, and Regulation Crypto Assets (Release 33-11434) is a proposal, not law [verified]. What follows is designed against both; it is not compliance with either.

5.3 Three things such an interface lists

The live Uniswap interface is the reference. It carries, in one surface:

1. **Ecosystem tokens, featured.** For LX, the set above: MOR, AI, ZOO, LUX.
2. **A launchpad long tail, aggregated.** Uniswap's Launches surface — an "all launchpads" filter, a trending list, "Pools is live: trade new Robinhood Chain tokens" — indexes tokens as launchpads mint them. The first target for LX is pools.trade on Robinhood Chain. Aggregation routes to the token's liquidity on its *source* chain; it does not bridge the token onto Lux and does not settle a security. In `lux/dex` the gateway already registers providers by priority; a launchpad is one more route-only provider, and adding the next one is a new registry entry, nothing else. [verified: lux/dex gateway provider registry; a route-only `LaunchpadProvider`]
3. **Tokenized equities, shown globally and geo-gated.** Uniswap lists NVDA under Stocks, "Issued by Robinhood," beside Coinbase, Xstock, Bstocks, and Ondo wrappers of the same underlying, and blocks the swap where the issuer's wrapper requires it: "NVDA unavailable in your region" [verified: live interface].

5.4 What we have, and what is switched off

Two codebases carry this lineage. `lux/exchange` is a debranded fork of the Uniswap interface; `lux/exchange2` is a ground-up, securities-native rewrite [verified: git]. Neither yet renders the Launches or Stocks surfaces; the histories are disjoint, so enabling them is a feature-directory port, not an upstream merge [verified]. The tokenized-equity taxonomy already exists — `AssetClass.STOCK` and an `Offering` union spanning `reg_cf`, `reg_a_plus`, `reg_s`, `reg_d`, and `rule_144a` [verified: asset-class.ts] — but the render is off: `Features.publicSecurities` defaults false, `canTrade` has no caller, and no equity appears in any token list. The taxonomy exists; the render does not [verified: equities.md, features.ts, regulated-provider.ts].

The binding constraint is not interface code. Uniswap's Launches and Stocks read its hosted data-api, which does not serve Lux chains. The real work is the data plane: index launchpad and tokenized-asset state for Lux, or aggregate it through the `lux/dex` gateway, which is that data plane. [verified: the lux/dex gateway connector is the source]

5.5 Two boundaries that do not move

Interface versus issuer.
Listing someone else's tokenized security is the Covered-User-Interface posture above. Issuing our own offering — a MOR raise, or any Reg CF, Reg A+, Reg S, or Reg D placement — is the issuer or portal role, a different and heavier one. Both are modeled in the `Offering` union; the ICO in Section 4 is the issuer path, gated on the proposed regulation [verified].

Aggregation versus bridging.
Featuring MOR, an OFT we bridge natively, is not the same as carrying a tokenized equity. An equity token keeps its issuer's non-US wrapper wherever it trades; the interface mirrors that restriction, it does not shed it by rendering. So the equity surface is shown globally, gated in the United States pending the rules now being finalized, and it ships only behind counsel. The flag flip is the client's, not ours — the render is off by a flag with no reader today, and the region block is net-new: only a UK information banner exists, so a fail-closed US block is a control to build, not toggle. [verified: features.ts, regulated-provider.ts, the UK-only region banner]

MOR is the shippable core today. The launchpad tail follows as the gateway connector lands. The equity surface is a built-but-gated capability whose switch is a legal decision, not an engineering one. One interface, three depths of review.

6 What we bring

This is the catalogue of what is available to adopt. Every entry is a real repository or paper on disk. Maturity is one of **verified live** (running in production on Lux, Hanzo, or Zoo, not yet on Morpheus), **built, not deployed** (code with tests, not activated on a live chain), or **research** (paper or proof, not shipped). One caveat is kept throughout: "gives Morpheus" means available to adopt, not already wired in. Nothing here settles a Morpheus session today.

6.1 Consensus and post-quantum

Quasar 3.0 (`lux/consensus`, `lux/quasar`; LP-020). DAG-native BFT that finalizes a block with one `QuasarCert` aggregating three lanes: a BLS12-381 classical fast path, a Corona Module-LWE threshold signature, and a Z-Chain verifier rollup. **Verified live** (LP-020 states activation on the Lux primary network 2025-12-25). For Morpheus, this makes the thin-MOR thesis concrete: a Morpheus L1 for session and settlement state can rent classical- fast and PQ-durable finality on the same certificate rather than build consensus.

Pulsar / Corona / Magnetar threshold family (`lux/pulsar`, `lux/threshold`; LP-073). Lattice threshold signatures with a lifecycle framework for permissionless validator sets; Pulsar (threshold ML-DSA) is an active NIST MPTC submission. **Built, not deployed** as a reusable primitive. For Morpheus, a threshold-sealed close-of-session receipt signed by a provider set rather than one hot secp256k1 key, which addresses the single-hot-key custody problem directly.

Unified PQ transport (`lux/papers` LP-202). One UDP port carrying QUIC over ZAP, an X25519MLKEM768 hybrid KEM in the TLS 1.3 handshake, hybrid classical and ML-DSA-65 peer identity. **Research** (specification). For Morpheus, the transport fix for the plaintext prompt path: prompts currently cross `:3333` in cleartext, and this is the drop-in PQ session layer.

P3Q unified-slot verifier (`lux/precompile/p3q`; LP-219). One EVM precompile slot, one ABI, a leading `kind` byte dispatching per-family threshold-signature verifier cores. **Built, not deployed**. For Morpheus, any contract, a settlement diamond included, can verify PQ threshold certificates on-chain without a bespoke precompile per scheme.

6.2 Verified inference and A-Chain

Inference precompile 0x0303 (`lux/precompile/inference`). A deterministic int8 transformer as an EVM precompile, pure Go with CGO=0, so every validator recomputes it byte-identically; GPU kernels are a KAT-proven byte-equal accelerator. **Built, not deployed**. For Morpheus, the missing settlement input: a deterministic small-model verification path is one honest way to bind payment to work for models small enough to reprice on-chain.

Attestation precompile 0x0301 (`lux/precompile/attestation`). Stateless TEE-quote verification (NVTrust GPU, TPM/SGX/SEV-SNP/TDX), chained to an embedded vendor root at the quote's own timestamp, fail-closed, no mutable device registry. **Built, not deployed**. This is the primitive behind Morpheus's own live TEE work (RTMR3, SecretVM), but as a consensus-checkable precompile rather than an off-chain check.

A-Chain inference bridge (`lux/precompile/aivmbridge`, `lux/precompile/modelregistry`). Two operations, submit-inference-intent (`modelSpecHash`, `promptHash`) and verify, plus a model registry precompile. **Built, not deployed**. For Morpheus, a `modelId` that is a registered, hashed model spec rather than a line in a local JSON file, which makes the resale substitution ECO-2 documents detectable.

Lux AI / A-Chain (`lux/ai`). OpenAI-compatible decentralized inference network, Q-Chain shared attestations, cross-network minting. **Built, not deployed** (our punch-list fixed a `lux/ai` unauthenticated RPC on 2026-08-29; see Section 8). For Morpheus, an existing inference-settlement chain to tap under thin-MOR rather than growing Morpheus into one.

Zoo proof-of-inference (`zoo/proofs`). A Freivalds matrix-product verifier over $\mathbb{F}_p$ ($p = 2^{61}-1$) that catches a fabricated product with probability at least $1 - 1/p$ per challenge, zero false rejection over an exact integer accumulator, wired to an activation trace so an operator cannot mint against an output it never computed. **Research** (proofs, some Lean-formalized). For Morpheus, a cheap probabilistic check that a claimed forward pass actually happened.

mord (`lux/mord`). A Lux L1 that runs an inference marketplace as chain state, separating capacity, work, and identity into three settlement quantities instead of Morpheus's one `pricePerSecond * elapsed`. **Built, not deployed** (our POC: 40 tests plus adversary tests, benchmarked at about 3,400 verified sessions/sec/core; not deployed, not audited; that number is settlement-rate, not network TPS). The reference design for what un-fakeable settlement looks like on their own market shape.

6.3 Agentic runtime

Enso learned router (`hanzo/enso`, ledger at `hanzo/ai/object/routing_event.go`). Classifies each request, prices it, routes to the best model across enabled providers, learns from in-app feedback and an LLM-as-judge signal, routes in microseconds on CPU. The ledger stores task label, routed model, token counts, latency, cost, an optional opaque feature vector, and a scalar reward, and stores no prompt text or hash. **Verified live** (powers `model=auto` on Hanzo Cloud). For Morpheus, a router that improves on a time-priced market and is the deployed proof that confidential routing is buildable: it consumes a low-dimensional statistic plus a scalar outcome, not content.

Hanzo agent SDK, agents, runtime (`hanzo/agent`, `hanzo/agents`, `hanzo/runtime`). Multi-agent orchestration over an MCP tool surface, role and workflow agents, and a sandboxed execution runtime for AI-generated code. **Built and verified live** (runtime ships releases). For Morpheus, the agentic execution layer that sits on sessions as the substrate for agent turns.

MCP tool surface (`hanzo/chat`, `mcp__hanzo__*`). Model Context Protocol tool use wired through `api.hanzo.ai/v1`. **Verified live**. Standard tool-calling for provider-side agents without inventing a protocol.

Hanzo Dev (`hanzo/dev`). A terminal coding agent, sandboxed repo edits, models over the metered gateway. **Verified live**. A reference consumer of metered per-session inference.

6.4 Confidential compute

FHE precompile (`lux/precompile/fhe`). CKKS via the pure-Go `luxfi/lattice`, threshold decryption through a T-Chain (LP-333). **Built, not deployed**. For Morpheus, compute over consumer data where the operator should not read inputs.

ZK, STARK-FRI, and anchor precompiles (`lux/precompile/zk`, `lux/precompile/starkfri`, `lux/precompile/anchor`; LP-221). Proof-verification and commitment precompiles, plus an anchor precompile that stores a Merkle root of CRDT state with per-app height monotonicity. **Built, not deployed**. The anchor primitive is exactly what the two-economy problem needs: Morpheus runs capital on Ethereum and compute on Base with a schedule replicated by hand (`SessionRouter.sol:58` says so [verified]), and anchoring lets one domain verify the other's state root instead of trusting a manual copy.

MPC custody stack (`lux/threshold`, `lux/kms`, `lux/mpc`). Universal threshold signing across many chains (CGGMP21 ECDSA, FROST Schnorr), MPC-backed KMS with per-org namespaces. **Built and verified live** (the canonical Lux KMS). For Morpheus, splitting the one provider hot key into capacity, work, and identity authorities with different blast radii.

6.5 Settlement, bridge, governance

Lux primary settlement and MOR OFT (`lux/node`, LayerZero OFT). MOR is already a live LayerZero OFT on Ethereum, Arbitrum, and Base. **Verified live**. For Morpheus, settle on a chain with real finality instead of application-layer storage on one Base diamond. The honest counterweight (LP-314) is that a sovereign L1 buys nothing worth the migration for much of the market, so this is an option, not a mandate.

Lux Bridge (`lux/bridge`). MPC cross-chain bridge, embeddable SDK, `api.bridge.lux.network`. **Verified live**. Native movement of MOR and stake across the capital and compute domains without a hand-replicated schedule.

l2-template and Quasar-rooted L1 (`lux` L2 template, `lux/evm`). An L2 stood up in minutes, rooted to Quasar PQ finality. **Built**. The minimal-effort route to a Morpheus-owned chain that inherits PQ finality rather than renting rollup execution.

Lux DAO and vote (`lux/dao`, `zoo/vote`). Governance contracts plus a UI, white-labeled per org. **Verified live** (`zoo/vote` deployed). An on-chain governance surface for parameter changes and treasury actions. Stated plainly: adopting a DAO UI does not by itself add a timelock or bound the mint; those are contract changes.

6.6 Fintech

LX order-book DEX (`lux/dex`). Pure-Go matching engine, order book, oracle aggregator, JSON-RPC/WebSocket/gRPC SDKs. **Verified live** (reference implementation runs standalone, ships releases). For Morpheus, real order-book depth for MOR against the thin AMM the protocol itself backstops.

Lux Exchange (`lux/exchange`). A Uniswap-forked front end plus a securities and ICO surface. **Built**; the securities/ICO path is **pending** SEC approval of a proposed regulation, a stated dependency and not a granted approval. Do not represent it as available.

Liquid self-repaying lending (`lux/liquid`; LP-310). Deposit yield-bearing collateral, borrow a 1:1-pegged synthetic, let yield retire the debt on a time-decay schedule; curated strategy adapters (Aave, Lido, Pendle, Morpho, Yearn). **Built, not deployed**. For Morpheus, an LMOR construction: self-repaying credit against MOR or against the roughly $20M stETH [verified] the treasury already holds, so holders borrow without selling into the $8.9k/24h [verified] volume.

Lux Bank (`lux/bank`, `lux/captable`). Accounts, cards, cap-table. **Built, not deployed**. A fiat and accounts on-ramp for the hosted-gateway audience, account-native consumers behind delegated sessions rather than one shared hot wallet.

6.7 Data and observability

GPU-native EVM and verification (`luxfi/evm`; LP-203). Unified C-Chain and L1 EVM plus GPU-native transaction verification. **Built** (EVM ships releases) and **research** (the billion-TPS figure is a paper claim, not a measured production number, labeled illustrative). Batched signature and proof verification throughput for a settlement chain. As with mord, the benchmark rate is not sustained network TPS.

Hanzo gateway, IAM, telemetry (`hanzo/gateway`, `hanzo/iam`). The trust boundary at `api.hanzo.ai` (JWT verified against IAM JWKS, client identity rewritten, rate limits, circuit breakers) and OIDC identity at `hanzo.id` (PKCE, SCIM, WebAuthn/MFA). **Verified live**. Morpheus's `api.mor.org` multiplexes many callers onto a few on-chain sessions behind spend limits that ship inert and are armed per environment; a verified-identity gateway with armed limits is the deployed answer.

Zoo Gym decentralized training (`zoo/gym`). An OSS LLM fine-tuning framework plus a protocol paper for ring-topology training across heterogeneous GPUs. **Built** (framework) and **research** (decentralized-training protocol). A provider-side path to sell training and fine-tuning yield, not only inference, on the same GPU fleet.

6.8 What this catalogue does not claim

Kept explicit. Nothing here settles a Morpheus session now; every "gives Morpheus" is an adoption, not a wiring. "Verified live" means live on Lux, Hanzo, or Zoo; the only Morpheus-side live PQ or TEE work is Morpheus's own, credited above. The billion-TPS and 3,400-sessions/sec figures are benchmark and rate numbers, not network TPS. The `lux/exchange` securities and ICO path is pending a regulatory approval that has not been granted. LP-312 through LP-326 are research papers grounded on Morpheus's deployed contracts; they are the argument for adoption, not evidence of it.

7 Building on their work

A migration that discards the good parts is not an improvement; it is a rewrite with a survivor bias toward whatever we happened to build. So the honest split is stated first, and it is short.

7.1 Keep

The attestation stack.
The 5,439-line TDX-to-TLS binding with RTMR3 in CI is real confidential-compute engineering. Keep it and give it a consensus-checkable home (attestation precompile 0x0301) so what today is an off-chain check becomes a settlement input. The local-attestation path in `lux/ai/pkg/attestation` removes the external NRAS and SecretVM availability coupling (SET-6); adopt it or keep their own, but keep the measurement work.

The bounty and audit design.
Real Code4rena, Zenith, and Renascence reports and a well-structured bounty program. Keep the process; it is the standard this evaluation holds itself to.

The treasury bifurcation.
Separate consumer and treasury payment paths with a 1% daily budget cap. It contained the July 2026 stipend leak. Keep the split; the fix that day-locked stipend on `min(closedAt, endsAt)` is exactly the right shape.

Receipt-derived accounting.
Stipend size and emission are pure functions, legible and independently checkable. Keep the receipt (it already signs both token counts) and read the two words the contract currently drops (SET-2).

The emission schedule.
14,400/day to zero in 2040, 42,000,000 terminal, no premine. The best property the token has. Keep it, and add the invariant cap the token lacks (SUP-1) so the property is enforced rather than conventional.

The community and the token.
MOR stays MOR. The migration keeps the ticker, the holders, the OFT deployment, and the schedule. Nothing here asks the community to move to a new token.

7.2 Replace

The settlement calculation.
`_getProviderRewards` pays elapsed time. Replace the payable event with a quorum-agreed receipt that commits to the output token count, so the fee base is reproducible (ADOPT-02). This is the one seam; everything above and below it stays.

The bare multisig.
A 5-of-9 with no timelock is the root finding. Replace it with a Safe plus Governor plus timelock, with a nonzero floor on the delay (ADOPT-03).

The buyback's fuel.
Deposit yield inverts when price falls. Replace it with a fee on verified volume (ADOPT-08), so the loop scales with usage.

The hand-replicated cross-domain schedule.
Replace the manual copy between Ethereum and Base with a native bridge and state-root anchoring (ADOPT-13, anchor precompile).

The self-signed ranking.
Replace the provider-signed `tps`/`ttft` scorer with a learned router keyed on an independent quality signal (Enso), which is orthogonal to and composes with quorum verification (SET-3).

8 Heal thyself

This section is first-person and unedited, because the pitch that Morpheus should adopt our stack to become real is dishonest until our own stack meets the standard we are selling. The rule it enforces: do not make a claim the code does not yet enforce. Every item below is a failure in our own tree (`luxfi`, `hanzoai`, `zooai`), verified against source on 2026-08-29. The decentralization study

grades six powers as a vector, and "theater" is the gap between a claimed state and the measured one. Each item moves a power off zero or closes a claimed-versus-measured gap.

8.1 Update: the on-chain governance path, built and proven (2026-08-30)

Since the audit above, the governance items (B3, B4, B5) moved from open to *built and demonstrated*. We stood the full stack up on a local network and exercised the decentralization end to end, verified on-chain, not described.

- A one-shot deploy brings up the entire stack — DEX, emissions, treasury, bridge, work-market, and the governance instrument — with every authority resting on a single 1/1 Safe: the honest bootstrap state [verified: local net].
- The governance instrument is the audited `DeployGovernance` wiring, and it comes up clean: the deployer renounces the Timelock admin (`deployer=false`, `self=true`), the Governor is the sole proposer, and the Safe is canceller/veto only. No residual deployer or EOA power [verified: on-chain reads].
- `lux.vote` is wired to the live Governor, and an admin dashboard (`admin.lux.cloud`) renders every governable surface with its current owner and a button that hands it from the Safe to the Timelock. We fired one such handoff through the 1/1 Safe on-chain — an AMM factory's authority moved Safe → Timelock, transaction status success [verified].

So the mechanism the pitch depends on — bootstrap under one key, then relinquish every parameter to token-holder governance — is no longer a proposal; it runs, and each surface it touches ends up controlled by the Governor through the Timelock, not by a key. That answers B3, B4, and B5 on the mechanism: the honest 1/1 bootstrap replaces the 3-of-5-from-one-mnemonic costume, the deployer-admin timelock is renounced by the correct script, and `lux.vote` over a live Governor replaces a branded front end over null contracts.

The claim we make, and the one we withhold. We claim: Lux has built and demonstrated, on-chain and end to end, an open on-chain governance system — `lux.vote` — and the path from a single bootstrap key to full token-holder control, with the Governor and Timelock live. We do not claim Lux is *already* fully decentralized on mainnet: today the mainnet parameters still rest on the bootstrap key, and the network reboot plus the parameter-by-parameter handoff to the Timelock are the executing step, not a finished one [pending]. Asserting the finished state before that handoff would be the exact theater we flag in Morpheus. So we state the measured position: the infrastructure is real and proven, the mainnet handoff is one execution away, and the same dashboard runs it.

8.2 The same twelve traps, in our own five systems

Before the punch-list, the mirror. The five economic and settlement failures we flag in Morpheus are not uniquely Morpheus's; we ran the same twelve-trap rubric over our own systems. Status is `present` (the defect exists and is reachable), `at-risk` (exists on a path not yet exploitable or a live design gap), `avoided` (closed in the reachable path, with the limit stated), or `n/a`. "papers" is the LP inference program read as a specification, so its `avoided` means closed by design, not deployed.

Trap	lux-ai	mord	hanzo-ai	gov/emission	papers
T1 unverified pay for claimed work	at-risk	**present**	at-risk	**present**	avoided
T2 no slashing of a bond	avoided	**present**	n/a	**present**	avoided
T3 uncapped mint	at-risk	avoided	n/a	at-risk	present
T4 owner upgrade, no delay	n/a	avoided	n/a	at-risk	at-risk
T5 no vote surface	n/a	avoided	n/a	at-risk	present
T6 owner-directed emission	n/a	n/a	n/a	at-risk	present
T7 prompt transport confidentiality	at-risk	n/a	at-risk	n/a	avoided
T8 resale indistinguishable	**present**	**present**	avoided	n/a	avoided
T9 revenue decoupled from volume	avoided	at-risk	avoided	**present**	present
T10 thin protocol-owned liquidity	n/a	n/a	n/a	at-risk	present
T11 consumer equals provider	at-risk	at-risk	avoided	**present**	present
T12 self-signed ranking	at-risk	at-risk	avoided	at-risk	at-risk

The sharpest admission is that the self-signed meter, the exact Morpheus defect, is `present` in mord (`market/settle.go:101-108`, the doc comment concedes "a Meter is a claim, not a proof" [verified]) and in the on-chain AI mint (`AIMining.submitProof` mints for a proof whose only checks are dedup, a timestamp window, and a hashcash difficulty test [verified]). mord closes T1 only when the aivm quorum hash replaces its meter as the settlement input; the fix already exists one directory over (`luxfi/ai/pkg/rewards` ships `SlashProvider` and `SettleQuorum`), and mord does not yet import it. We are proposing to sell verification we have not yet made the default in our own systems. Until at least mord's T1/T2 and `lux/ai`'s RPC path are closed, a competent Morpheus reviewer reading our tree would find we ship the same defect. The punch-list below is the work that makes the pitch honest.

8.3 Blocking, before the pitch

B1, DONE 2026-08-29.
`lux/ai`'s served RPC was unauthenticated and returned a fabricated completion. Fixed: an `authMiddleware` (constant-time bearer check, fails closed when no key is set) now gates chat, embeddings, and task-submit; the chat handler refuses (503/501) instead of fabricating a completion; `handleSubmitResult` rejects results for unknown tasks, so no unassigned work is credited. Tests in `cmd/lux-ai/auth_test.go` pass. The canonical quorum settlement path (`chains/aivm`) remains authoritative; the placeholder no longer lies about it.

B2.
vMOR is flat 1:1, not duration-weighted as an earlier note of ours claimed (`VotesERC20StakedV1.sol:407` mints `amount` directly, gating only on one global `minimumStakingPeriod`). Fix: implement Curve-style duration weighting, or delete the claim (already corrected in the token-design note), and set quorum above the owner-directed emission share so the vote cannot be captured by emission recipients.

B3.
The `lux/dao` launch Safe is 3-of-5 from one mnemonic; effective control set is one (`governance.md:22-28`, `LUX_MNEMONIC`). Fix: rotate to independent hardware keys held by distinct named people, publish the proofs, and move `owner()` of the DAO modules to the token-controlled timelock.

B4.
The `lux/standard` production timelock keeps the deployer as sole proposer and admin, never renounced (`DeployMultiNetwork.s.sol:249-254`). Fix: renounce deployer-admin; move the proposer role to the Governor.

B5.
The reference tenant `zoo/vote` points at null contracts (`config/zoo.json`, governor and factory addresses null). Fix: deploy the governor set and fill the config, or stop citing `zoo/vote` as a live governance instance. A branded front end over no contracts is theater.

8.4 Required for a real end-state

R1.
The `l2-template` chain is centrally sequenced at birth, single-replica MPC, KMS, and bridge, with an uncapped-mint admin token (`init.sh:240,181,184`). Fix: distribute the validator set to independent operators, split MPC/KMS/bridge across distinct operators, cap or remove the generated mint, and measure the realized Nakamoto coefficient rather than asserting one.

R2.
Uncapped mint in `MOROFT` and the generated `Token.sol` (`MOROFT.sol:50,61`). Fix: for any chain we control, cap or remove the mint. For Morpheus, the same fix is ADOPT-03 and SUP-1 on their side.

R3.
The Lux primary genesis seats about five stakers and five bootstrappers, reward-forfeiture only, no

principal slashing. Permissionless entry exists in code (`AddPermissionlessValidatorTx`); the live set is the gap. Fix: open the producer set to independent operators and publish the distribution.

R4.
mord is a single-operator PoC (`mord/README.md:5-6`). It is an integrity improvement, not a decentralization claim; deploy it under a validator set before calling it decentralized. Honest as disclosed.

8.5 Commitments, the ones that cost us something

These are not code. They are what we must agree to, or the "open governance" claim is theater regardless of the contracts. We do not hold a controlling share of the vote, and if any reward flow routes through us we disclose it and cap our weight below a control threshold. We cannot both direct emission and claim we do not control governance. Permissionless validator entry must exist in the live set, not only in the code. Keyholders are named people with published proofs. No retained privileged upgrade path survives once the token-controlled timelock is authoritative.

Every failure above is disclosed in our own code or configs, which by the study's own standard makes it immaturity, not theater. The discipline is simple: do not make a claim the code does not yet enforce. Where we are early, we say early. This punch-list is the difference between shipping that discipline and shipping a better-looking version of what we are asking Morpheus to leave behind.

9 The path

Three phases, with a gate between each. A gate is a measured condition, not a date. The migration is phased and reversible, and the seam is one layer: replace the settlement engine, keep everything above and below it.

9.1 Phase 0: on Base, no migration, reversible

Deploy the four one-transaction fixes (ADOPT-09): route ownership through a timelock (CTL-1), decode all seven receipt fields and bind the approval address (SET-2, SET-5), and add the self-deal check (SET-4). Deploy verified settlement as an adjunct to the existing diamond (ADOPT-02): attestation-gated registration and a quorum check that must confirm before `_getProviderRewards` pays. Stand up a demand source and attested supply (ADOPT-07): Hanzo as a first attestable provider through the same config-only backend path any provider uses, with Enso routing behind `api.mor.org`. Providers keep their nodes; consumers see no change. This phase closes CTL-1, SET-1 through SET-5, ECO-3, and half of SET-3.

Gate to Phase 1: the timelock owns the proxies and the diamond; verified settlement confirms before payment on a non-trivial share of live sessions; a named demand source is producing measurable settled volume.

9.2 Phase 1: governance, finance, bridge

Deploy governance (ADOPT-03): Safe plus Governor plus timelock with a nonzero delay floor, and a vMOR vote surface, closing CTL-2 and moving the seat of control off the bare multisig. Route the buyback's fuel to a fee on verified volume (ADOPT-08). List MOR on the LX order book to deepen a book that is 88.8% protocol-owned (ADOPT-06). Deploy LMOR self-repaying credit against the idle stETH and MOR (ADOPT-04), and the bank rail for fiat-native consumers (ADOPT-05). Bridge MOR natively across the capital and compute domains and anchor state roots rather than replicating the schedule by hand (ADOPT-13). MOR the token, its emission, and its capital stay on Ethereum and Arbitrum; the only change to the token is the buyback's additional source.

Gate to Phase 2: governance parameters sit behind the timelock with a floor above zero; the fee-fed buyback is live and its dollar flow is published against emission; settled verified volume is large enough that a few sessions per second is a binding constraint.

9.3 Phase 2: a chain of its own, thin MOR, emission cut

Only if the Phase 1 gate is met. Move settlement onto a Lux L1 or L2 (ADOPT-01) that inherits Quasar PQ finality rather than building consensus, with MOR bridged as gas and mord as the

reference VM; keep MOR thin on top of it (ADOPT-12); lift the throughput ceiling (ADOPT-11). Run the ICO on `lux.exchange` conditional on the regulatory approval [pending], to fund the transition and deepen liquidity, not to substitute for demand. Introduce xMOR as a share-token and cut emission, both of which move the same net-supply inequality from Section 4. One provenance note we carry from our own audit: ML-KEM is live on Lux mainnet, while ML-DSA and SLH-DSA are present but not activated, so post-quantum signatures are a Phase 2 claim, not a current fact.

10 What each party gets, and what we must commit to

The migration is a Morpheus governance decision. We cannot execute it, and we should not write or speak as though the choice is ours. What follows is what each party gets if the community chooses it, and, stated with equal weight, what we must commit to for it to be real rather than theater.

Morpheus gets

- A token that earns from volume rather than deposit yield, with the emission schedule, the holders, and the ticker unchanged.
- Settlement that reads the work its receipt already signs, so payment matches the whitepaper.
- Governance with a delay and a vote where today there is a bare multisig.
- Post-quantum consensus, verified inference, threshold custody, and a native bridge, inherited rather than built by a five-person team.

Hanzo gets

- The demand and routing layer (`api.mor.org` plus Enso), and, running attested inference, the largest verified provider on the network.

Lux gets

- The settlement, verification, and transport layer, and the settlement fee for securing it.

Zoo gets

- A live governance tenant: MOR holders voting on a real economy through the DAO stack.

What we must commit to, or none of the above is real

- Close our own punch-list first. mord and the on-chain AI mint currently reproduce the exact Morpheus settlement defect in our own code; until at least mord's verified path and `lux/ai`'s RPC path are the default, we are selling verification we have not made our own default. B1 is done; the rest are not.
- Name a demand source and a volume before claiming the buyback math works. The earning model converts demand into token value; it does not manufacture demand. If V is zero, no take-rate saves it.
- Do not hold a controlling share of the vote, and disclose and cap any reward flow that routes through us.
- Say early where we are early. Every "verified live" in this document means live on Lux, Hanzo, or Zoo, not on Morpheus, and every research paper is an argument for adoption, not evidence of it.

The honest obstacle is convincing existing providers to bond stake and accept slashing where today they bear none, and convincing the multisig to hand parameters to a timelock. Phase 0's reversibility, and the fact that it turns on real revenue, are the argument that has to carry the vote. We supply the engine and the proof. The Morpheus community chooses whether to run it.